

Computer Systems Short Note

Contents

1	Data Representation and Computer Arithmetic	1
1.1	Data Representation	1
1.1.1	Decimal Numbers	1
1.1.2	Octal Numbers	1
1.1.3	Hexadecimal Numbers	1
1.1.4	Binary Numbers	2
1.2	Computer Arithmetic	2
1.2.1	Representing Numbers	2
1.2.2	Negative Numbers in Binary	2
1.2.3	Scientific Notation	4
1.2.4	Non-Integer (Fraction) Number Conversion	4
1.2.5	Floating Point Representation	5
2	Basic logic circuit design	7
2.1	Introduction to Boolean Algebra and Logic Operators	7
2.1.1	Boolean Algebra	7
2.1.2	Logic Operations	7
2.1.3	Forms of Boolean Expressions	9
2.1.4	Boolean Expressions and Truth Tables	10
2.1.5	Boolean Algebraic Laws	12
2.1.6	Karnaugh Maps	13
2.1.7	Single Gate Type Circuits	13
2.2	Combinational Logic Circuits	13
2.2.1	Adder	14
2.2.2	Subtractor	15
2.2.3	Multipliers	17
2.2.4	Decoder and Encoder	17
2.2.5	Multiplexer	18
2.3	Sequential Logic Circuits	19
2.3.1	Latch	20
2.3.2	Flip-Flop	21
3	Inner workings of CPU	24
3.1	The CPU and its Organization	24
3.1.1	Key Contents	24
3.1.2	Organization of CPU	25
3.1.3	Main Memory	25
3.1.4	Memory Controller	25
3.1.5	Bus	26
3.1.6	Clock	27
3.2	Inner Workings of CPU	28
3.2.1	Von Neumann Architecture	28
3.2.2	The Fetch-Decode-Execute Cycle	28
3.2.3	Instruction Set Architecture (ISA)	28

3.2.4	Addressing Modes	30
3.2.5	Instruction Level Pipelining	31
4	Memory Components and organization	33
4.1	Primary and Secondary Memory	33
4.2	Types of Memory	33
4.3	Memory Hierarchy	34
4.4	Typical Data Access Point	34
4.5	CPU Memory Gap	34
4.6	Principal of Locality	35
4.7	Cache Memory	35
4.7.1	Key Design Decisions of a Cache	36

1 Data Representation and Computer Arithmetic

1.1 Data Representation

1.1.1 Decimal Numbers

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Converting Decimal to Binary
 - Keep dividing the decimal number by 2 until the remainder is 0, and concatenate the quotients as the last quotient is the most significant bit.
- Converting Decimal to Octal
 - Keep dividing the decimal number by 8 until the remainder is 0, and concatenate the quotients as the last quotient is the most significant bit.
- Converting Decimal to Hexadecimal
 - Keep dividing the decimal number by 16 until the remainder is 0, and concatenate the quotients as the last quotient is the most significant bit.

1.1.2 Octal Numbers

- 0, 1, 2, 3, 4, 5, 6, 7
- Converting Octal to Decimal
 - Multiply the place value with the 8's power of the bit value ($8^0, 8^1, 8^2 \dots$) add all of them together.
- Converting Octal to Binary
 - Convert single digits of octal numbers into binary, by using the Converting Decimal to Binary, and concatenate the 3 digit binary numbers together.
- Converting Octal to Hexadecimal
 - Convert single digits of octal numbers into binary, by using the Converting Decimal to Binary, and concatenate the 3 digit binary numbers together.
 - Then use the method of converting binary to hexadecimal to find the hexadecimal equivalent.

1.1.3 Hexadecimal Numbers

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- Converting Hexadecimal to Decimal
 - Multiply the place value with the 16's power of the bit value ($16^0, 16^1, 16^2 \dots$) add all of them together.
- Converting Hexadecimal to Binary
 - Convert single digits of octal numbers into binary, by using the Converting Decimal to Binary, and concatenate the 4 digit binary numbers together.
- Converting Hexadecimal to Octal
 - Convert single digits of octal numbers into binary, by using the Converting Decimal to Binary, and concatenate the 4 digit binary numbers together.
 - Then use the method of converting binary to octal to find the octal equivalent.

1.1.4 Binary Numbers

- 0, 1
- Converting Binary to Decimal
 - Multiply the place value with the 2's power of the bit value ($2^0, 2^1, 2^2...$) add all of them together.
- Converting Binary to Octal
 - Separate the binary number into groups of 3 starting from the least significant bit.
 - Convert the 3 bit numbers into decimal and concatenate them together.
- Converting Binary to Hexadecimal
 - Separate the binary number into groups of 4 starting from the least significant bit.
 - Convert the 4 bit numbers into decimal and concatenate them together.

1.2 Computer Arithmetic

1.2.1 Representing Numbers

- Unsigned representation: non-negative integers
 - First bit is the MSB.
 - Number is stored in binary.
 - Only positive numbers can be stored.
- Signed representation: integers
 - First bit is reserved for sign bit.
 - 0 for positive and 1 for negative numbers.
 - Number is stored in binary.
- Floating point representations

1.2.2 Negative Numbers in Binary

- Sign-magnitude Representation.
 - Sign bit is flipped.
 - Benefits with sign-magnitude representation:
 - * Easy for understanding.
 - Problems with sign-magnitude representation:
 - * Two zeros. (00000000 and 10000000)
 - * Need to consider both sign and magnitude for calculations.
 - Calculations:

$$\begin{aligned} &= 5 + (-3) \\ &= 00000101 + 10000011 \\ &= 00001000 \\ &= 8 \end{aligned}$$

- One's Complement

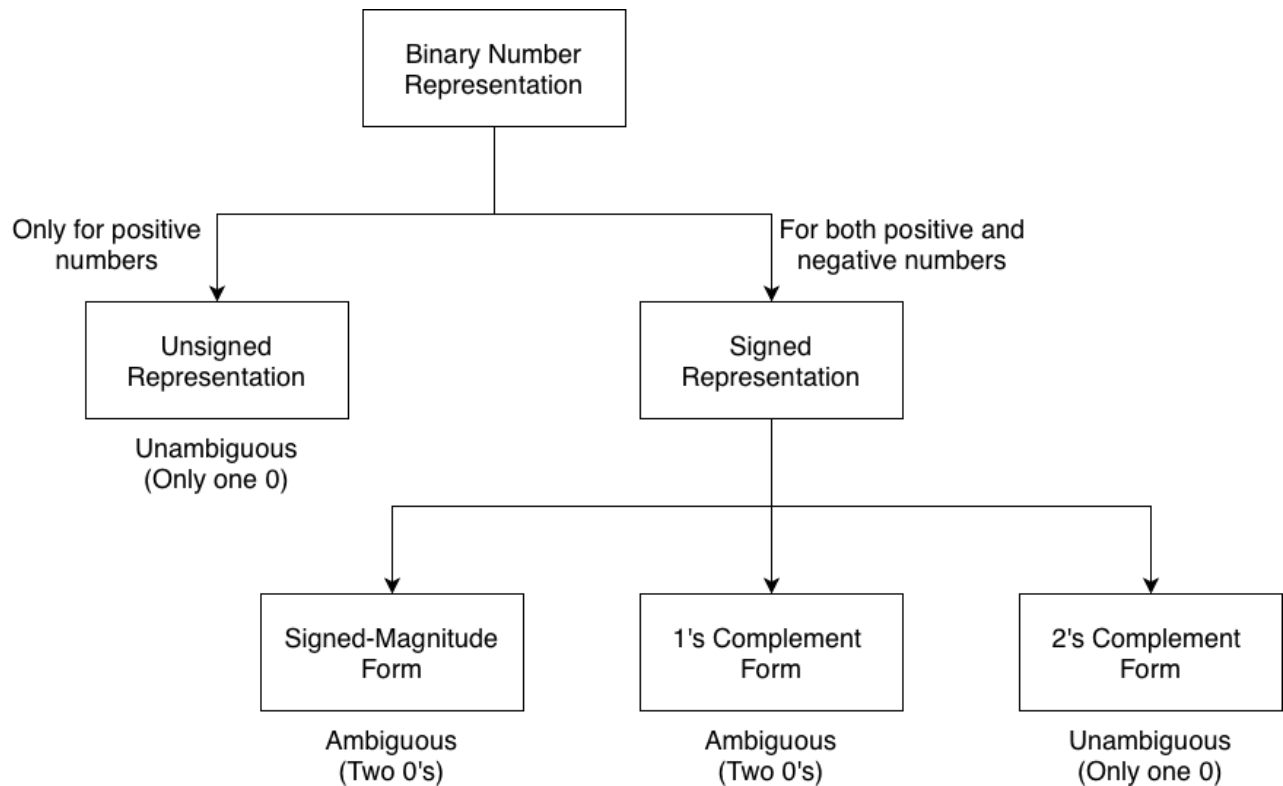
- Positive numbers remains unchanged.
- Negative numbers, the binary value's bits are flipped.
ex. $1001 \rightarrow 0110$
- Since positive numbers always begins with a 0 then the flipped bit makes the negative number begins with a 1.
- Benefits with One's Complement:
 - * Easy to negate.
 - * Subtraction can be performed as addition.
- Problems with One's Complement:
 - * Two zeros (00000000 and 11111111).
 - * Carry around is generated (Not clean calculation).
- Calculations:

$$\begin{aligned}
 &= 5 + (-3) \\
 &= 00000101 + 11111100 \\
 &= 1|00000001 \\
 &= 00000001 + 00000001 \text{ (Carry Bit)} \\
 &= 00000010 \\
 &= 2
 \end{aligned}$$

- Two's Complement

- Positive numbers remains unchanged.
- Negative numbers, the binary value's bits are flipped and another 1 is added.
ex. $1001 \rightarrow 0110 + 1 \rightarrow 0111$
- Two's complement is one's complement + 1.
- Benefits with Two's Complement:
 - * One zero.
 - * Subtraction and addition are identical processes.
- Problems with Two's Complement:
 - * Since there is only 1 zero the range is asymmetric (for 4 bits the range is +7 to -8).
 - * Finding the negative number is complicated since it requires bit-wise inversion and an increment.
- Calculations:

$$\begin{aligned}
 &= 5 + (-3) \\
 &= 00000101 + 11111101 \\
 &= 1|00000010 \text{ (Additional bit is ignored)} \\
 &= 00000010 \\
 &= 2
 \end{aligned}$$



1.2.3 Scientific Notation

- Decimal
 - $1.3526 * 10^7$
 - * 3526 → Mantissa (Significand)
 - * 10 → Base
 - * 7 → Exponent
- Binary
 - $1.001 * 2^{-5}$

1.2.4 Non-Integer (Fraction) Number Conversion

- Converting decimal numbers with fractions to binary:
 - 16.375
 - Convert 16 in the normal way.
 - $16 \rightarrow 10000$
 - For the 0.375, continue to multiply it by 2 until the 375 becomes 0. The numbers you get next to the decimal point each time will be the binary representation of the number.

$0.375 * 2$
 $0.750 * 2 \rightarrow 0 \& 0.750$
 $1.500 * 2 \rightarrow 1 \& 0.500$
 $1.000 \rightarrow 1 \& 0.000$
 $\therefore 0.375 \rightarrow 0.011$
 - $\therefore 16.375 \rightarrow 10.011_2$

- Converting binary numbers with fractions to decimal:

- 10.111
- Convert integer part in the normal way
- $10 \rightarrow 2$
- for 0.111 use the reciprocal value of the bit index and add the multiplications of the reciprocal values together.

$$(1 * \frac{1}{2^1}) + (1 * \frac{1}{2^2}) + (1 * \frac{1}{2^3})$$

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8}$$

$$\therefore 0.111 \rightarrow 0.875$$

$$\therefore 10.111_2 \rightarrow 2.875$$

1.2.5 Floating Point Representation

- 32 bit representation.

←1 bit→	←8 bits→	←23 bits→
---------	----------	-----------

- First bit is reserved for the sign.
- Second 8 bits are for biased exponent.
- Last 23 bits are for the mantissa (significand).
- To find the biased exponent, add the exponent to the maximum number can be made with the given bits.

ex. Biased exponent is represented 8 bits. Maximum number can be made is 127 ($2^8 - 1$).

Let the exponent is 5,

$$\text{Biased exponent} = 127 + 5 = 132.$$

- Example:

263.3 as 32-bit binary floating point number.

$$263 \rightarrow 100000111_2$$

$$0.3 \rightarrow .0100110011001101_2$$

$$263.3 \rightarrow 100000111.0100110011001101_2$$

$$\rightarrow 1.000001110100110011001101 * 2^8$$

Signed bit = 0 (number is positive)

Exponent = 8

$$\text{Biased Exponent} = 8 + 127 = 135 = 10000111$$

Mantissa = 00000111010011001100110 (LSB is ignored to fit into a 23 bit space)

0	1	0	0	0	0	1	1	1	0	0	0	0	1	1	1	0	1	0	0	1	1	0	0	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- Normalization:
 - Explicit Normalization:
 - Move the radix point to LHS of the MSB.
 - Normally used when the binary number looks as bellow.
 - $0.00110_2 \rightarrow 0.110 * 2^{-2}$
 - Implicit Normalization:
 - Move the radix point to RHS of the MSB.
 - Normally used when the binary number looks as bellow.
 - $101.100_2 \rightarrow 1.01100 * 2^2$
- Limitations:
 - Floating point representation only an approximate real number.
 - Overflow and underflow can cause programs to crash:
 - * Floating Point Overflow:
 - Floating point representation can overflow when the exponent exceeds the maximum number possible with the given bits.
 - If biased exponent is represented using 8 bits, the exponent value cannot exceed 127.
 - $1.110101 * 2^{128}$ cannot expressed using this floating point representation.
 - $1.110101 * 2^{128} \approx \infty$
 - * Floating Point Underflow:
 - Floating point representation can underflow when the exponent drops bellow the minimum number possible with the given bits.
 - If biased exponent is represented using 8 bits, the exponent value cannot be less than -126.
 - $1.110101 * 2^{-127}$ cannot expressed using this floating point representation.
 - $1.110101 * 2^{-127} \approx 0$
 - Rounding Off Error:
 - * When smaller floating-point number is shifted to make the exponent equal, some of the least significant bits are lost.
 - * This loss of precision is called rounding.
 - * Rounding-by-chop : If the left over is less than 0.5, it is chopped after the mantissa's maximum digit position.
 - * Rounding-to-nearest : If the left over is greater than 0.5, then add 1 to the LSB and carry if necessary.

2 Basic logic circuit design

2.1 Introduction to Boolean Algebra and Logic Operators

2.1.1 Boolean Algebra

- There are only two states,

0 if false

1 if True

- Basic operations

– AND $\rightarrow A \cdot B$

– OR $\rightarrow A + B$

– NOT $\rightarrow \bar{A}$

- Boolean Function

$$f(x, y) = x.\bar{y} + \bar{x}.(x + y)$$

- Truth Table

$$F = x + y.\bar{z}$$

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

2.1.2 Logic Operations

- AND Operation

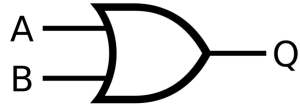
– $A.B$ or $A \wedge B$



A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

- OR Operation

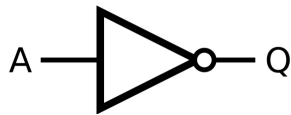
– $A + B$ or $A \vee B$



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

- NOT Operation

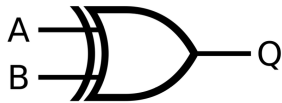
– $\overline{A}$ or $\neg A$



A	Q
0	1
1	0

- XOR Operation

– $A \oplus B$



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

- NAND Operation

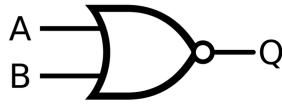
– $\overline{A.B}$



A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

- NOR Operation

- $\overline{A + B}$



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

- XNOR Operation

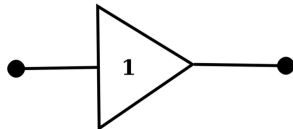
- $\overline{A \oplus B}$



A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

- Buffer Operation

- A basic logic gate that passes its input, unchanged, to its output.
 - Used to increase the propagation delay in circuits.



A	Q
0	0
1	1

2.1.3 Forms of Boolean Expressions

- Canonical Form

- Expression must contain all the variables in the function.
 - Sum of Minterms (Standard Products)
 - * Variables bound by AND operator is called a minterm.
 - Ex. $F(x, y) = x.y + \bar{x}.y + \bar{x}.\bar{y}$
 - Product of Maxterms (Standard Sums)
 - * Variables bound by OR operator is called a maxterm.
 - Ex. $F(x, y) = (x + y).(x + \bar{y}).(\bar{x} + \bar{y})$

- Standard Form
 - Expression do not necessarily contain all the variables in the function.
 - Sum of Products
 - Ex. $F(x, y) = x.y + \bar{x}$
 - term $\bar{x}$ do not contain the variable y
 - Product of Sums
 - Ex. $F(x, y) = \bar{y}.(x + y)$
 - term $\bar{y}$ do not contain the variable x

2.1.4 Boolean Expressions and Truth Tables

- Truth Table to SOM

Ex:

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1 $\bar{x}.y.\bar{z}$
0	1	1	0
1	0	0	1 $x.\bar{y}.\bar{z}$
1	0	1	1 $x.\bar{y}.z$
1	1	0	1 $x.y.\bar{z}$
1	1	1	1 $x.y.z$

$$F(x, y, z) = \bar{x}.y.\bar{z} + x.\bar{y}.\bar{z} + x.\bar{y}.z + x.y.z$$

- Truth Table to POM

Ex:

x	y	z	F
0	0	0	0 $x + y + z$
0	0	1	0 $x + y + \bar{z}$
0	1	0	1
0	1	1	0 $x + \bar{y} + \bar{z}$
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F(x, y, z) = (x + y + z).(x + y + \bar{z}).(x + \bar{y} + \bar{z})$$

- SOM to Truth Table

Ex:

$$F(x, y) = \bar{x}.\bar{y} + x.\bar{y} + x.y$$

x	y	F
0	0	1 $\bar{x}.\bar{y}$
0	1	0
1	0	1 $x.\bar{y}$
1	1	1 $x.y$

$$F(x, y) = \Sigma(0, 2, 3)$$

x	y	F
0	0	1 $\bar{x}.\bar{y}$
0	1	0
1	0	1 $x.\bar{y}$
1	1	1 $x.y$

- POM to Truth Table

Ex:

$$F(x, y) = (\bar{x} + \bar{y}).(x + \bar{y}).(x + y)$$

x	y	F
0	0	0 $x + y$
0	1	0 $x + \bar{y}$
1	0	1
1	1	0 $\bar{x} + \bar{y}$

$$F(x, y) = \Pi(0, 2, 3)$$

x	y	F
0	0	0 $x + y$
0	1	0 $x + \bar{y}$
1	0	1
1	1	0 $\bar{x} + \bar{y}$

2.1.5 Boolean Algebraic Laws

Annulment Law

$$A.0 = 0$$

$$A + 1 = 1$$

Identity Law

$$A + 0 = A$$

$$A.1 = A$$

Idempotent Law

$$A + A = A$$

$$A.A = A$$

Complement Law

$$A + \overline{A} = 1$$

$$A.\overline{A} = 0$$

Commutative Law

$$A.B = B.A$$

$$A + B = B + A$$

Double Negation Law

$$\overline{\overline{A}} = A$$

Distributive Law

$$A.(B + C) = A.B + A.C \quad A + (B.C) = (A + B).(A + C)$$

Redundancy Law

$$A + A.B = A$$

$$A(A + B) = A$$

De Morgan's Law

$$\overline{A.B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A}.\overline{B}$$

2.1.6 Karnaugh Maps

- A gate level minimization technique.
- Grouping Rules:
 - Groups should not contain different types.
 - Diagonal grouping is not allowed.
 - Groups can only contain 2^n number of cells, where $n \geq 0$.
 - Each group should be its maximum size.
 - Groups may overlap when they are of maximum size.
 - Groups may be formed around the table.
 - There should be as few groups as possible.
- If a function is incompletely specified, meaning there are input patterns which do not generate any output patterns, these are called don't care conditions in K-maps.
- Those don't care patterns are marked with a X and considered as 0 or 1 when grouping to form the most optimal groupings.

2.1.7 Single Gate Type Circuits

- Circuits are preferred to be made using single gate type.
- This makes the ICs cost effective.
- There can be unused gates in ICs, if they were to be made with different gates.
- Functional Completeness:
 - AND, OR and NOT are the primary functionalities.
 - Therefore any gate type that can demonstrate all three primary functionalities are called Functionally Complete Set.
 - By its definition {AND, OR, NOT} is a functionally complete set.
 - Other Functionally Complete Sets;
 - {AND, NOT}
 - {OR, NOT}
 - {NOR}
 - {NAND}
 - NAND and NOR are identified as individually functionally complete.
 - They can create any boolean function without any other gates. Therefore NAND and NOR are called Universal Gates.

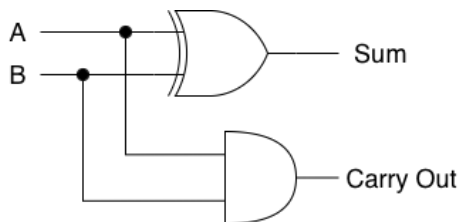
2.2 Combinational Logic Circuits

- The output is purely depend only on its current inputs.
- Purely built on logic gates.
- No feedback paths, No memory elements.

2.2.1 Adder

- Half Adder:

- Use to perform addition of two bits.
- Half adder cannot handle the carry out bits within it self.
- Therefore it has two outputs as sum and carry out.



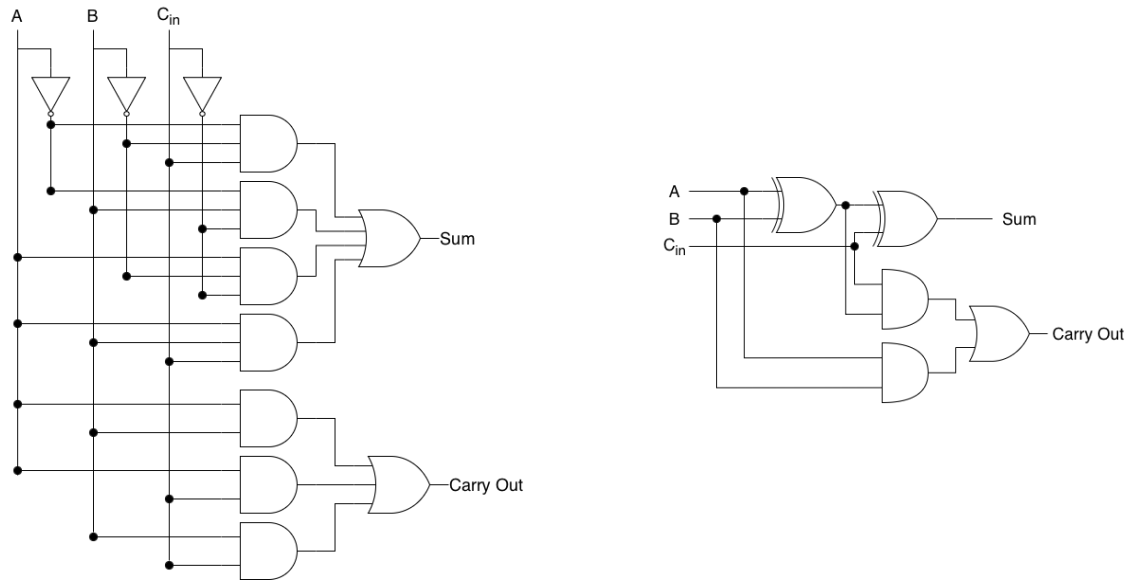
A	B	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- Sum = $A \oplus B$
- Carry Out = $A.B$

- Full Adder:

- Use to perform addition of three bits.
- Carry out of the previous calculation is taken in as an input to the next calculation.
- Full adder can be implemented by combining two half adder circuits.

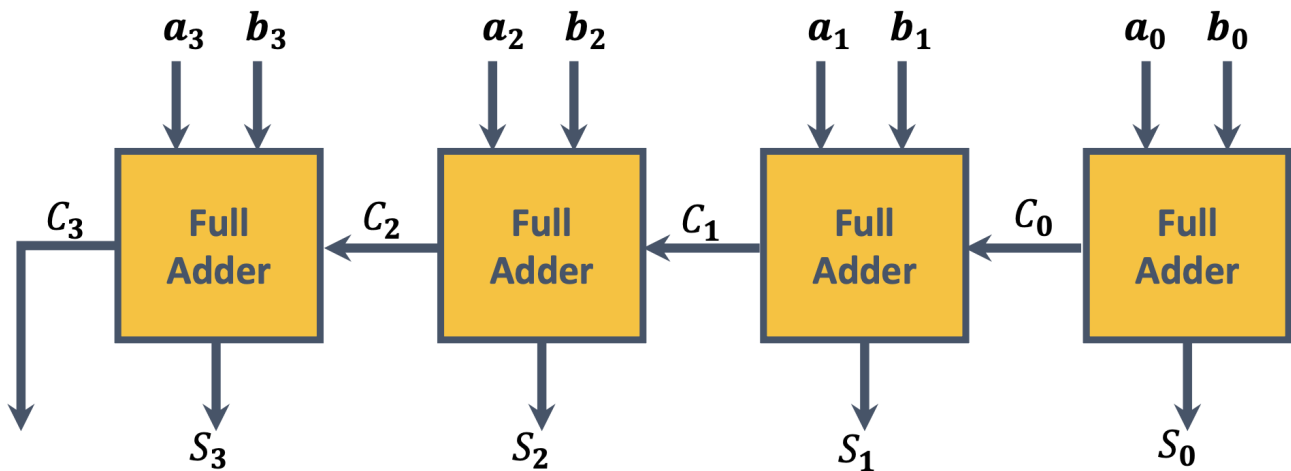
A	B	C_{in}	Sum	Carry Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



- $\text{Sum} = \overline{A}.\overline{B}.C_{in} + \overline{A}.B.\overline{C_{in}} + A.\overline{B}.\overline{C_{in}} + A.B.C_{in} \rightarrow \text{Sum} = A \oplus B \oplus C_{in}$
- $\text{Carry Out} = \overline{A}.B.C_{in} + A.\overline{B}.C_{in} + A.B.\overline{C_{in}} + A.B.C_{in} \rightarrow \text{Carry Out} = A.B + A.C_{in} + B.C_{in}$

- Ripple Carry Adder:

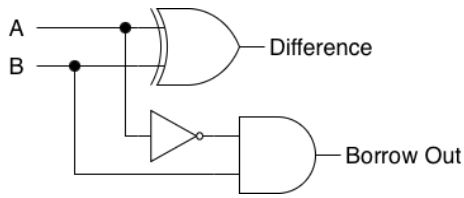
- Use full adders to add multi bit binary numbers.
- For n bits require n full adders.



2.2.2 Subtractor

- Half Subtractor:

- Use to perform subtraction of two bits.
- Outputs the difference of the bits and the borrow out.
- Cannot handle borrowing ins.
- The borrow out signal is set when the subtractor needs to borrow from the next digit in a multi-digit subtraction.



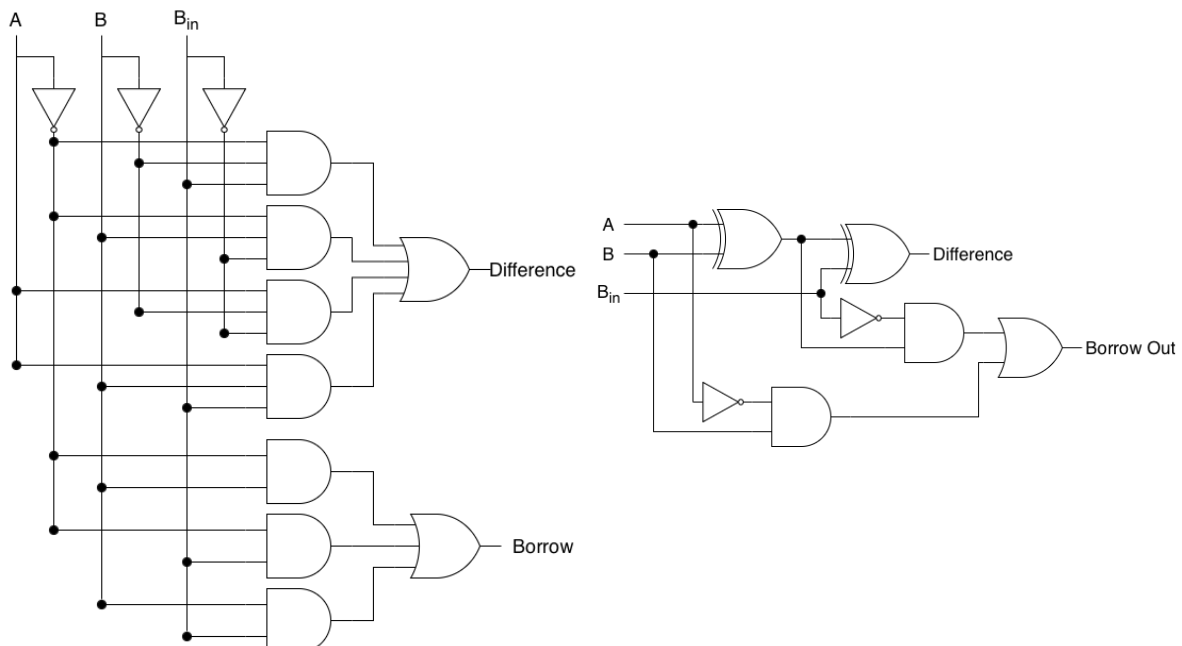
A	B	Difference	Borrow Out
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

- Difference = $A \oplus B$
- Borrow Out = $\bar{A}.B$

• Full Subtractor:

- Use to perform subtraction of 3 bits.
- Borrow out of the previous calculation is taken in as an input to the next calculation.
- Full Subtractor is implemented by combining two half subtractor circuits.

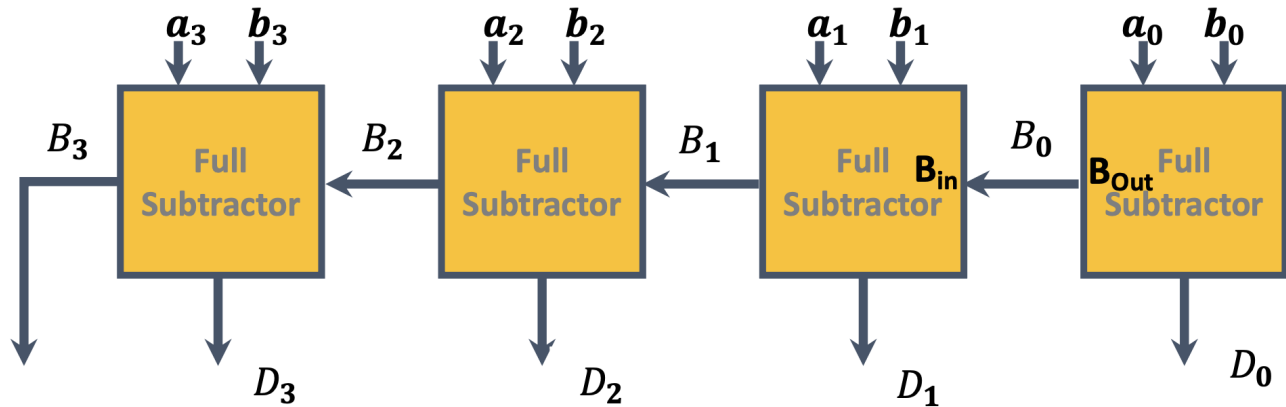
A	B	B_{in}	Difference	Borrow Out
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



- Difference = $\overline{A}.\overline{B}.B_{in} + \overline{A}.B.\overline{B_{in}} + A.\overline{B}.\overline{B_{in}} + A.B.B_{in} \rightarrow \text{Difference} = A \oplus B \oplus B_{in}$
- Borrow Out = $\overline{A}.\overline{B}.B_{in} + \overline{A}.B.\overline{B_{in}} + \overline{A}.B.B_{in} + A.B.B_{in} \rightarrow \text{Borrow Out} = \overline{A}.B + \overline{A}.B_{in} + B.B_{in}$

- Ripple Borrow Subtractor:

- Use full subtractors to find the difference between multi bit binary numbers.
- For n bits require n full subtractors.

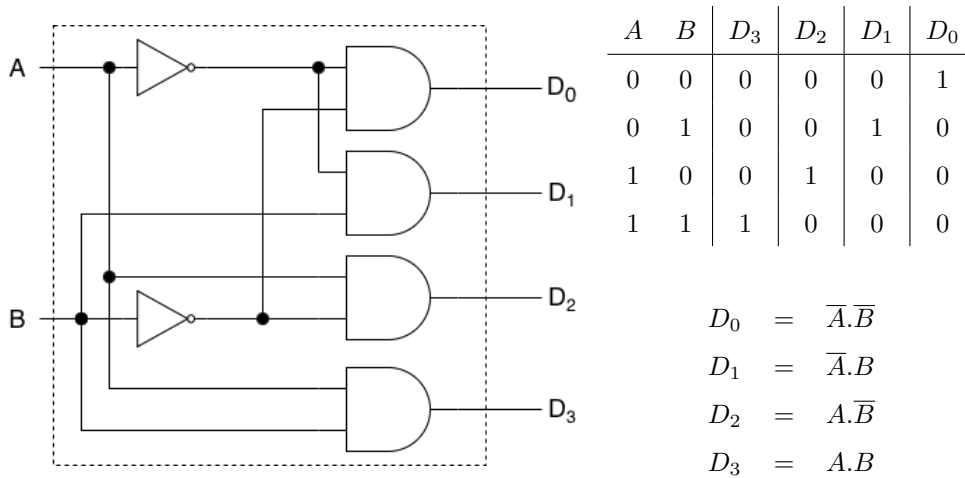


2.2.3 Multipliers

- Use to perform multiplication of two bits.
- Process occur in two steps, just like in long multiplication:
 - Generation of partial products: Multiply on of the numbers using all the bits in the other number.
 - Summation of partial products: Add all the partial products while shifting them one bit to left.
- Architecture:
 - AND gates are used to generate the partial products.
 - Full Adders are used to the summation of shifted bits.
 - Physical wiring make sure the shifting process.

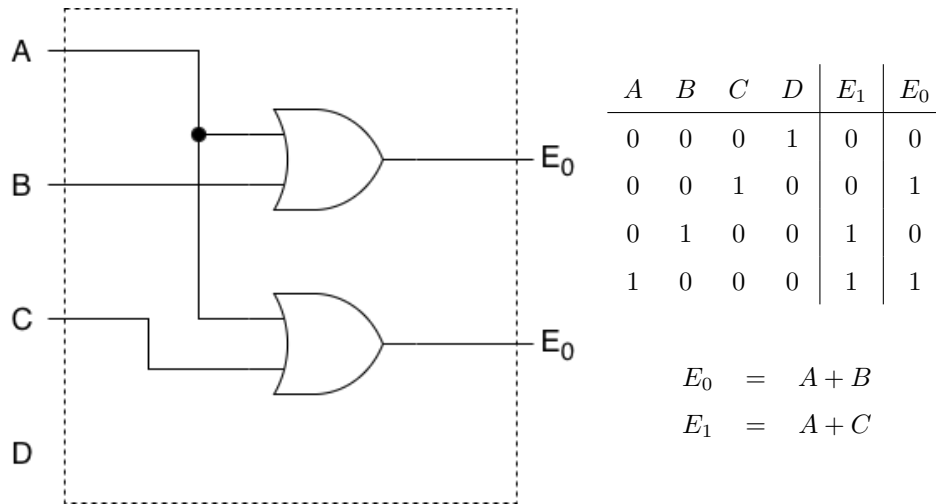
2.2.4 Decoder and Encoder

- Decoder:
 - Decoder is a special circuit unit to intrepret a given code word.
 - For n input lines, the circuit can support up to 2^n unique output lines.
Ex. 2 input and 4 (2^2) output channels $\rightarrow$ 2 x 4 Decoder.
 - Most decoders include an "Enable" (E) input. If the Enable pin is set to 0, the entire circuit is effectively turned off, and all outputs remain 0.
 - Decoders can be used as circuit builders since they convert any input function into Sum of Minterms form. Logical sum can be provided with an external OR gate.



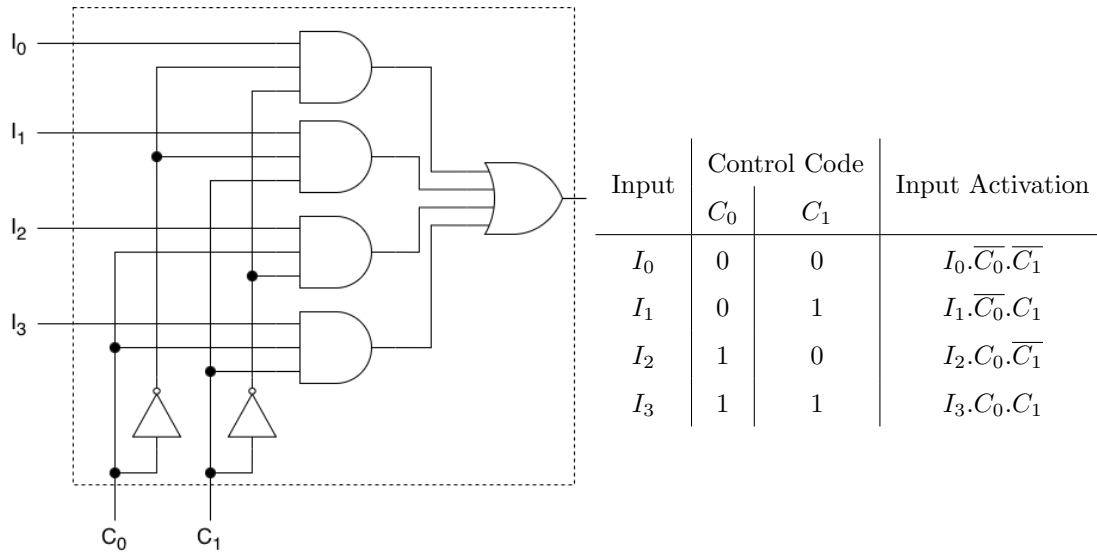
- Encoder:

- Performs the reverse operation of Decoder.
 - There should be 2^n input channels for n output channels.
- Ex. 4 (2^2) input and 2 output channels $\rightarrow$ 4 x 2 Encoder.



2.2.5 Multiplexer

- Multiplexer is a fundamental digital circuit with multiple inputs and a single output.
- It's main purpose is to select a single signal coming from the input lines and forward it to the output line.
- The circuit consists of several control lines to direct an input channel to the output line.
- Dedicated control line pattern as the pass code for each input.
- There should be n number of control lines for 2^n input lines.

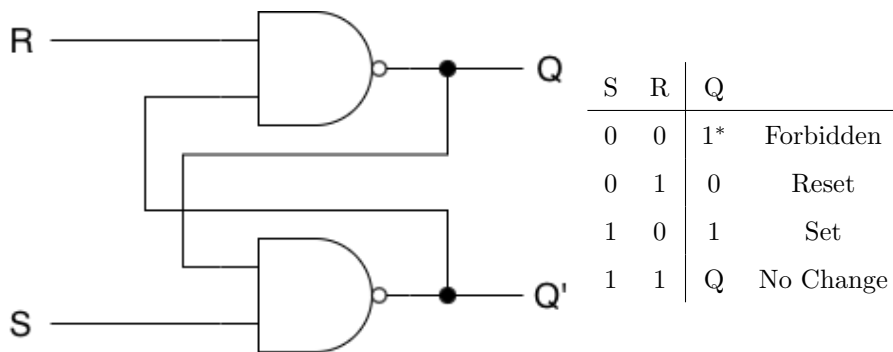
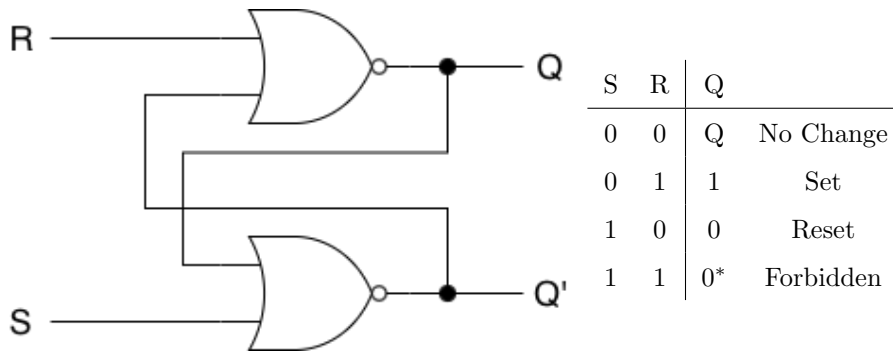


2.3 Sequential Logic Circuits

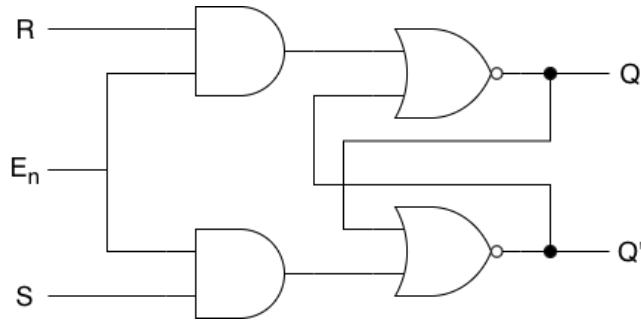
- The output depends on present inputs as well as the previous outputs that are stored in an memory element.
- Memory element's transition among states.
- The next state of the memory element is determined by its input and the present state.
- A circuit can be seen as a time sequence of inputs, outputs and internal states.
- Synchronous Sequential Circuits :
 - The state changes only at discrete time intervals, controlled by a global clock signal.
 - All memory elements update their values simultaneously on either rising or falling of clock pulse.
 - Advantages :
 - * Easy to design since all components change state at the same time.
 - * Hazards and race conditions are minimized since operations have to wait for a clock pulse.
 - * Easy to make complex circuits like CPU because timing constraint is global.
 - Disadvantages :
 - * Slower in action since the clock speed is determined by the slowest path in the circuit.
 - * High power consumption due to distribution of clock signal throughout the circuit.
 - * In large systems it is difficult to ensure clock signal reaches all the components at the same instant.
- Asynchronous Sequential Circuits :
 - The state of the circuit changes immediately after the input changes.
 - No global clock is used.
 - Advantages :
 - * Faster since the absence of global clock speed, the circuit operates at the maximum speed allowed by it's components.
 - * Low power consumption since only the components involves in the operation consumes energy.
 - * Problem with synchronizing the clock signal is eliminated.
 - Disadvantages :
 - * Harder to design.
 - * Circuits may be unstable since different signals may arrive at different times.

2.3.1 Latch

- A latch is transparent. This means that as long as the enable signal is active (high), the output will constantly track the input. If the input changes while the enable signal is high, the output changes immediately.
- Has the ability store one bit.
- Latches are level sensitive. Therefore they do not operate on clock edges and is an asynchronous device.
- A clock can be used as a control signal in latches. When control signal is active latches make a transition.
- But if the inputs have changed during clock pulse active period, then the latch will generate two outputs during a single clock pulse.
- **S-R Latch :**
 - The SR Latch is the most basic building block of memory. It has two main inputs Set (S) and Reset (R).
 - Generate two outputs and one is the complement of the other.
 - When $Q \geq 1$ Set state and otherwise Reset state.

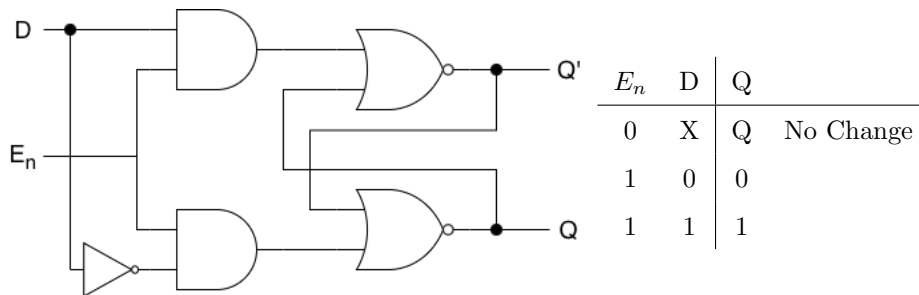


- S-R Latch can work with control signals. In the below circuit E_n should be triggered to make the S-R Latch operational.



• D Latch :

- The D Latch is an improvement on the S-R Latch designed to solve the "invalid state" or "forbidden state" problem. It has a single D (Data) input and an Enable (E_n) input.
- Value of D goes out only when the circuit is enabled and when disabled it remains unchanged.

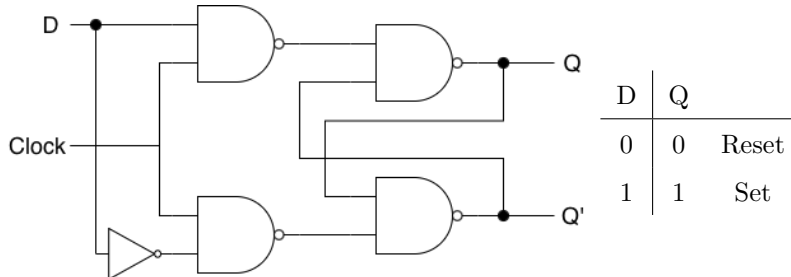


2.3.2 Flip-Flop

- A flip-flop is designed to be more stable than a latch. Flip-flops are made with edge sensitivity.
- Instead of responding to the level of the clock signal, it responds only to the transition (the edge) of the clock signal—either from 0 to 1 (rising edge) or 1 to 0 (falling edge).
- Positive Edge (Rising Edge) :
- Negative Edge (Falling Edge) :
- Solves the problem where the Latches generating two outputs when used with a clock.
- Has the ability store one bit.

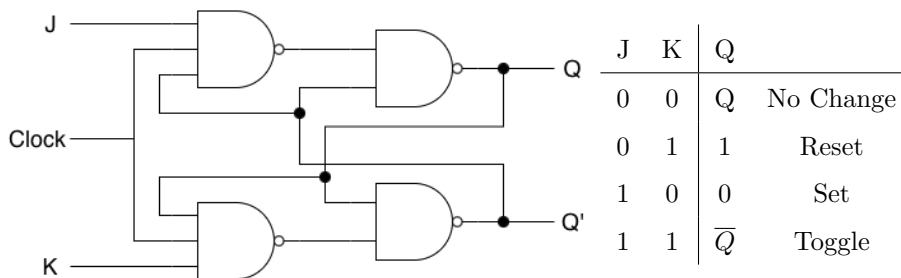
- **D Flip-Flop :**

- The D Flip-Flop is the most widely used in digital systems.
- It acts as a simple buffer that captures the value on the D (Data) input and transfers it to the output Q at the next clock edge.



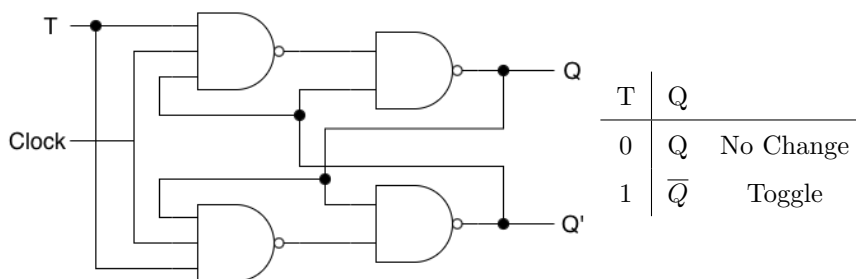
- **J-K Flip-Flop :**

- The J-K Flip-Flop is an advanced version of the S-R Latch.
- It solves the "invalid state" problem of the S-R latch by adding a feedback mechanism.



- **T Flip-Flop :**

- The T Flip-Flop is a simplified version of the J-K flip-flop where the J and K inputs are tied together.



- Applications of Flip-Flops:
 - Registers :
 - * Registers are a series of flip-flops with a common clock input.
Ex. 32 bit registers are made with 32 flip-flops.
 - * Combinational logic circuits may also combine with registers to aid additional operations.
 - * Storage Registers :
 - This is the simplest register, which is implemented by using series of D flip-flops.
 - Binary input is stored at the flip-flop using a clock signal, and can be read at any time.
 - It is possible to clear all the flip-flops using a separate asynchronous input signal.
 - * Shift Registers :
 - A type of sequential digital circuit used for storing and moving data.
 - It is a chain of D flip-flops connected together so that the output of one flip-flop serves as the input to the next on every clock edge.
 - Random Access Memory (RAM) :
 - * The most basic form of RAM, SRAM (Static RAM), is built using a latch or flip-flop structure.
 - * Without the ability to "remember" its state over time, RAM would essentially just be a set of switches that reset to nothing every time you turned them off.
 - Counter :
 - * A counter is one of the most classic examples of a sequential circuit because its output at any given moment is determined by its current count (the stored state) plus the input (the clock pulse).
 - * Ripple Counters :
 - In a ripple counter, only the first flip-flop is connected to the external clock. The output of each flip-flop serves as the clock input for the next one.
 - The first flip-flop changes state on the clock edge. Its output then acts as the clock for the second flip-flop, causing it to change state shortly after. This continues down the chain like a wave or a "ripple" in water.
 - Built using D flip-flops.
 - * Synchronous Counters :
 - Synchronous counters are the high-performance counterparts to ripple counters. In these circuits, every flip-flop is triggered simultaneously by the same global clock signal.
 - Because the clock is connected to every flip-flop at once, all flip-flops change state at the exact same moment. There is no "rippling" effect, making them much faster and more reliable than asynchronous designs.
 - Built using J-K flip-flops.

3 Inner workings of CPU

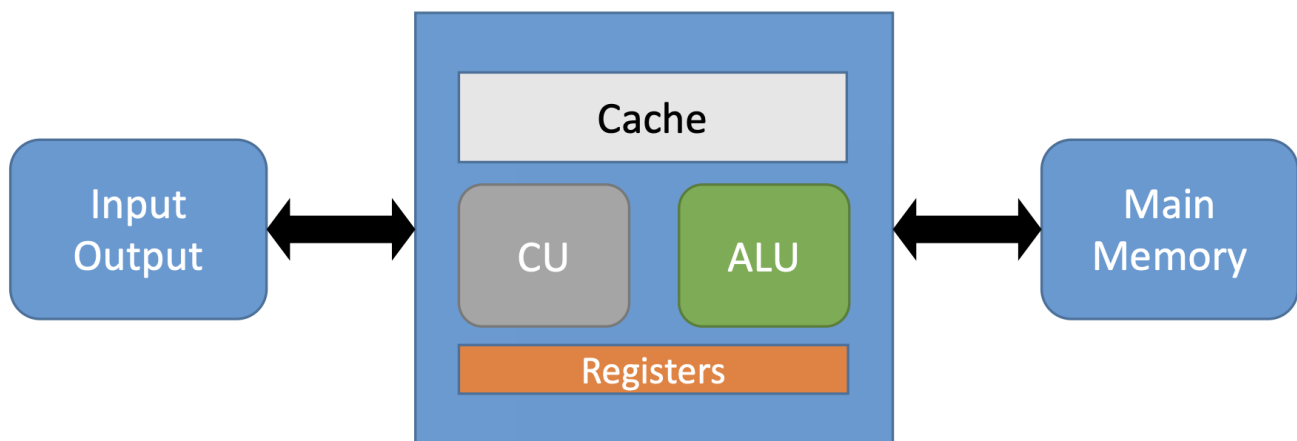
3.1 The CPU and its Organization

3.1.1 Key Contents

- ALU (Arithmetic Logic Unit)
 - Perform mathematical and logical operations
- CU (Control Unit)
 - Fetch instruction from the main memory
 - Decode instruction
 - Determine what operations the ALU should perform
 - Control the data flow
- Registers
 - Storage medium that can store binary information
 - Faster memory access
 - Store wide variety of data such as Instruction, Data and Address.
 - Key registers,

MAR	Memory Address Register	Hold the address of location of data to be accessed
MDR	Memory Data Register	Hold data that being transferred to or from the memory
AC	Accumulator	Stores intermediate arithmetic and logic results
PC	Program Counter	Contains the address of next instruction to execute
CIR	Current Instruction Register	Contains the current instruction while processing

- Cache
 - Store most frequently used data.
- Input/Output Interfaces



3.1.2 Organization of CPU

- Control Unit (CU and Registers)
 - Manages the control flow of data and instructions
- Execution Unit (ALU and Cache)
 - Performs operations of data

3.1.3 Main Memory

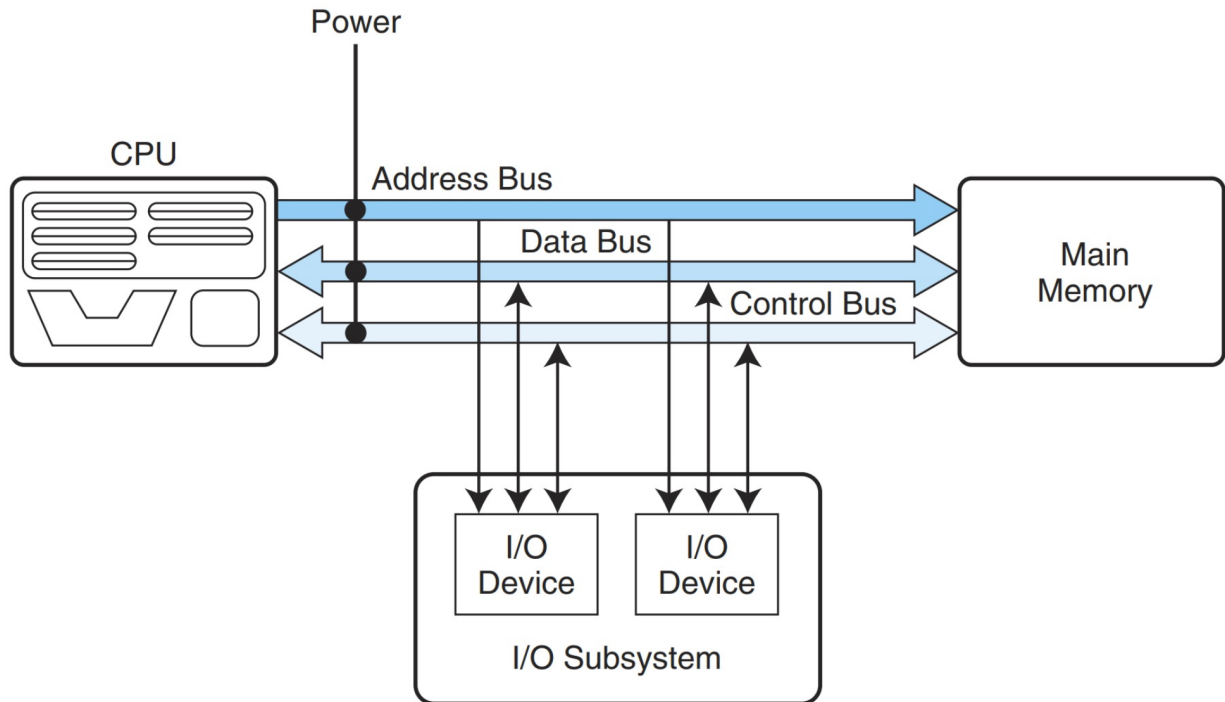
- Data is stored in binary
- Contains blocks of data where the smallest memory unit is a bit
- Group of bits, is called a word, with fixed size of data.
- Each word has a unique address.
- Different attributes of Main Memory
 - Address length
 - Address range
 - Word size
 - * Byte Addressable
 - Addressing a memory location byte by byte (8 bits), instead of referring to a whole word.
 - If N is the memory address it points to a byte (block of 8 bits) in a memory so N + 1 points to the next byte.
 - Most used in modern computers.
 - * Word Addressable
 - Addressing a memory location word by word (32 or 64 bits).
 - If N is the memory address it points to a word (block of 32 or 64 bits) in a memory so N + 1 points to the next word.
- Maximum number of addresses in a memory,
 2^n , where n is the address length in bits.
- Maximum memory capacity,
 $2^n * (1 \text{ byte or } 1 \text{ word})$, where n is the address length in bits.

3.1.4 Memory Controller

- Manages the data flow that goes in and out of the computer memory.
- Coordinate communication between CPU and main memory.
- Managing tasks as error checking and correction.

3.1.5 Bus

- A bus is a connection of wires that connects multiple subsystems as a path for data to travel.
- Only one device can use the bus at a given time, since sharing the bus creates bottlenecks.
- Bus speed depend on the width of the bus and no. of devices that shares it.



- Address Bus
 - Sends the address of the location of data the CPU wants to write or read.
 - One Directional
- Data Bus
 - Sends the data.
 - If CPU sent a read request, the data is sent from main memory to CPU and if CPU sent a write request, the data is sent from CPU to main memory.
 - Bi Directional
- Control Bus
 - Sends the control signals.
 - Can include read and write commands, interrupt requests, and others.
 - Bi Directional

3.1.6 Clock

- Generate ticks (clock pulses).
- Pulse sets the pace of every operation.
- Unit of measurement is Hz.
- Clock speed (Clock rate) - Number of pulses per second
- Clock cycle time
 - The period of the clock that synchronize with the circuit of a process.

$$\text{Clock Cycle Time} = \frac{1}{\text{Clock Speed}}$$

- Cycles per instruction (CPI)
 - The average number of cycles per instruction.

$$CPI = \Sigma (CPI * F)$$

For each instruction its individual CPI and Frequency.

- No. of Instructions
 - The static instruction count is wrong since it doesn't count all the instructions the CPU executes.
 - Correct measure is the dynamic instruction count.
- CPU Clock Cycles

$$\text{No. of Clock Cycles} = N * CPI$$

N is the number of instructions.

- Clock execution time
 - CPU execution time of a program.

$$T = \text{No. of Clock Cycles} * \frac{1}{\text{Clock Speed}}$$

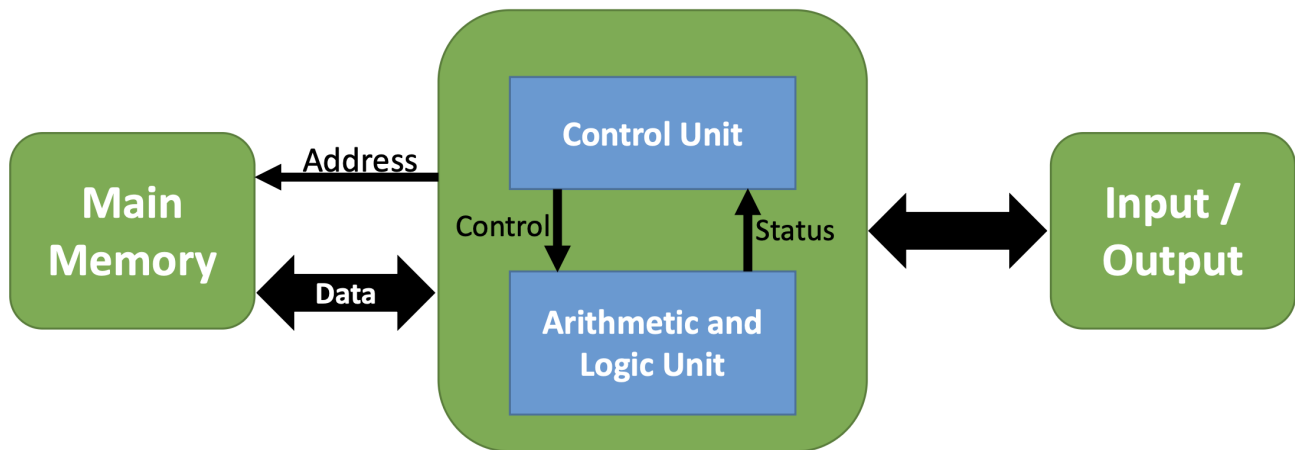
$$T = \text{No. of Clock Cycles} * \text{Clock Cycle Time}$$

$$T = N * CPI * \text{Clock Cycle Time}$$

- CPU throughput is impacted by the following factors,
 - No. of instructions
 - No. of Cycles per instruction
 - Clock cycle time

3.2 Inner Workings of CPU

3.2.1 Von Neumann Architecture



3.2.2 The Fetch-Decode-Execute Cycle

- CPU fetch the instruction to execute next from memory. The instruction is loaded to the necessary register.
- CPU decodes the instruction. The required operand are fetched from memory and stored in the registers
- CPU executes the instruction.
- CPU update the program counter.
- Repeat from the top.

3.2.3 Instruction Set Architecture (ISA)

- A set of machine specific instruction that computer can perform and the format of each instruction
- Consider a machine architecture with 256 byte main memory, 16 general registers, 16 bit instructions, 8 bit integers and floating points.
 - Opcode (The instruction) will take 4 bits from the instruction.
 - Since there are 16 registers, one register address will be represented as 4 bits.

- Data Transfer Instructions

- Transfer data between registers and memory cells

L	R,	A		Load the register R with the content of memory cell with address A
ST	R,	A		Store the content in register R on the memory cell with address A

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Opcode				Register				Memory Address							

LI	R,	I		Load the register R with the immediate number I
----	----	---	--	---

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Opcode				Register				Immediate number							

LR	R1	R2		Load register R1 with the content on register R2
----	----	----	--	--

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Opcode				Unused				Register				Register			

- Arithmetic / Logical Instructions

- Perform arithmetic or logical operations

ADD	R0,	R1	R2		Add the content on R1 and R2, then store the answer in two's complement on R0
AFP	R0,	R1	R2		Add the content on R1 and R2, then store the answer in floating point on R0
OR	R0,	R1	R2		OR bit pattern of R1 and R2, then store the answer in R0
AND	R0,	R1	R2		AND bit pattern of R1 and R2, then store the answer in R0
XOR	R0,	R1	R2		XOR bit pattern of R1 and R2, then store the answer in R0

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Opcode				Register				Register				Register			

- Control Instructions
 - Control the execution of the program

JMP	R, A	Jump the instruction located on the memory cell with address A, if the content on R equals to content on R0
-----	------	---

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Opcode				Register				Memory Address							

HALT	Halt the execution
------	--------------------

3.2.4 Addressing Modes

Addressing modes specify where the instruction operands are located.

- Effective address
 - The location of the actual address of an operand is called the effective address.
 - In a computer with a virtual memory, the effective address is the virtual address or register.
 - In a computer with a main memory, the effective address is the main memory address or register.
- Physical address
 - The actual physical address is invisible to a programmer due to MMU.
 - Memory Management Unit is responsible of handling the actual physical address since it has access and control over the main memory
- Common Addressing Modes
 - Immediate addressing

LI R1, I

- * Pros.
 - Faster execution since the data is part of the instruction.
- * Cons.
 - Value is fixed during the compilation process.
 - Length of the value has to be fit inside the available address field.

- Direct addressing

L R1, A

- * Pros.
 - Fast because value can be accessed in a single reference.
 - Flexible than immediate addressing.
- * Cons.
 - It provides only a limited address space.

- Indirect addressing

LD R4, (R3)

- * Brackets indicate the pointer value for the register R3.
- * Like the pointer operation in C.
- * Pros.
 - Available a large memory space since entire word can hold an address.
- * Cons.
 - Requires to get two reference from memory. One to memory address and one to its value.

- Register direct addressing

ADD R0, R1, R3

- * Similar to direct addressing.
- * Instead of memory a register is used to specify operand.

- Register indirect addressing

- * Similar to indirect addressing.
- * Operand specifies register which has a pointer to memory block with data.

- Displacement addressing

- * Combination of direct addressing and register indirect addressing.
- * Operand specifies register which has a pointer to memory block with data.

- Stack addressing

3.2.5 Instruction Level Pipelining

- Modern CPUs divide the fetch-decode-execute cycle into small parts.
- Each steps are made independent so they can run parallel.
- Pipeline Efficiency:

- Consider a pipeline with k stages.
- The clock cycle time is t_p .
- There are n instructions to be executed.
- If the current instruction is I_1 , then its execution time is $k * t_p$.
- The other instruction will require a time of $(n - 1) * t_p$.
- Therefore the total time spend is equal to,

$$(k * t_p) + ((n - 1) * t_p) = (k + n - 1) * t_p$$

- Pipeline Speedup:

- If we were to compare the speed gain from pipelining compared to a normal CPU,

$$Speedup = \frac{k * t_p * n}{(k + n - 1) * t_p}$$

- Advantages:

- Better CPU utilization.
- Higher throughput.
- In a well-balanced pipeline, the time spent on each stage is nearly the same, therefore once the pipeline fills one instruction is complete.

- Disadvantages:
 - If a certain stage lags behind, the CPU will be idle for some time.
 - If depended instructions are met, it will force a instruction to stop until the other completes.
- Limitations:
 - Consider we keep increasing the number of stages,
 - The overhead in managing process.
 - Hardware complexity grows.
 - Clock speed gain reduces.
- Issues:
 - Pipeline stalls (waiting for a lagging stage)
 - Flushes (throwing away partially done works)
 - Hazard handling complexity.

4 Memory Components and organization

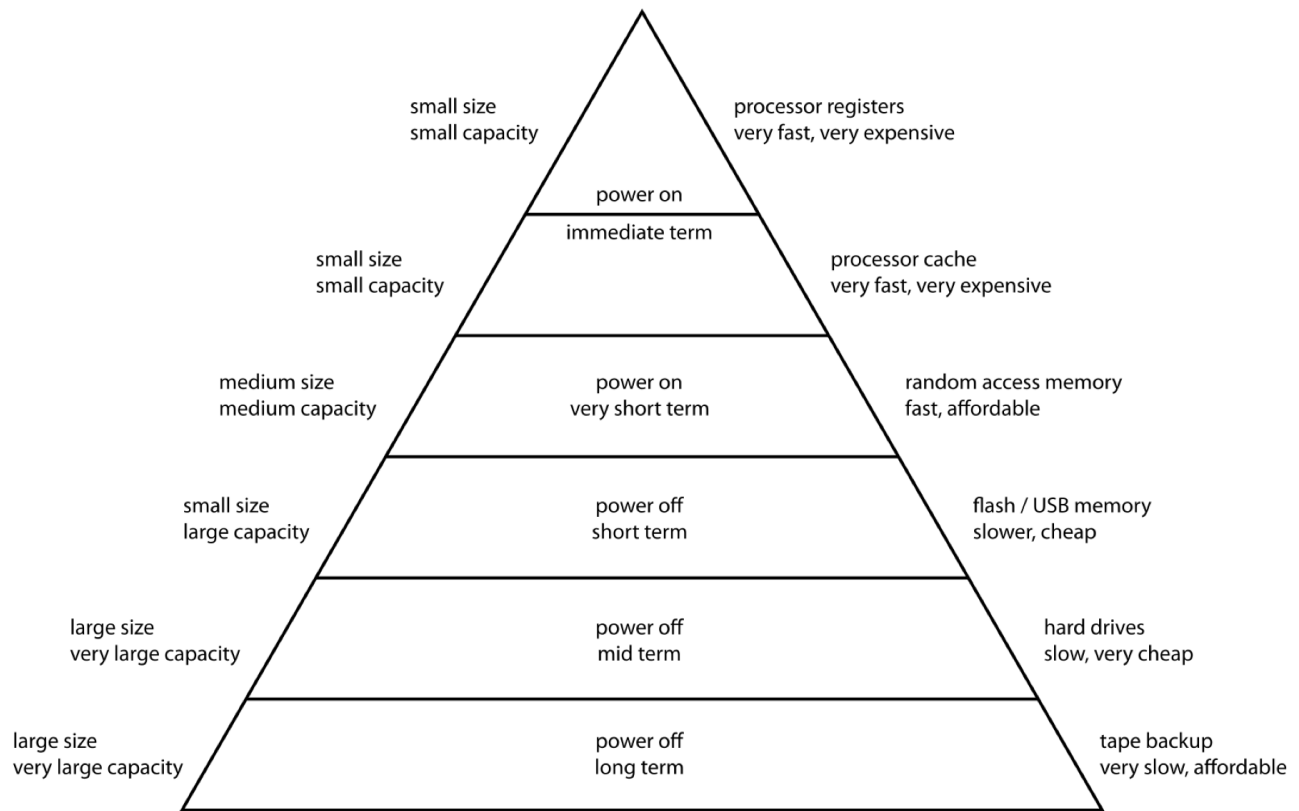
4.1 Primary and Secondary Memory

- Primary Memory
 - Cache Memory
 - Main Memory
- Secondary Memory
 - Magnetic Disks
 - Magnetic Tapes
 - USB Drives

4.2 Types of Memory

- Read Only Memory (ROM)
 - Non Volatile
 - Variants
 - * PROM - Programmable Read Only Memory
 - * EEPROM - Electrically Erasable Programmable Read Only Memory
- Random Access Memory (RAM)
 - Volatile
 - Variants
 - * DRAM - Dynamic Random Access Memory
 - * SRAM - Static Random Access Memory
- Flash Memory
 - Non Volatile
 - Read and Write
 - Common Usage
 - * Memory Cards
 - * Solid State Drives
 - * USB Flash Drives

4.3 Memory Hierarchy



4.4 Typical Data Access Point

- CPU checks data from the nearest storage (Cache).
- If not found, it then checks in main memory.
- If not found, it then checks in HDD.
- Once data is found, it will be brought into main memory and then into cache.

4.5 CPU Memory Gap

- Memory access time cannot match the CPU speed.
- HDD > SSD > Main Memory > Cache >> CPU
- This gap effects the CPU perform.
- "Principal of Locality" is used to bridge the gap.

4.6 Principal of Locality

- Principal of locality is a feature in a program on how memory references are grouped together by time and space.
- Temporal Locality (Time Locality)
 - This states that if a memory block is referenced once, it is most likely to be accessed in the near future.
 - Since loops uses the same variable multiple times.
 - Therefore caching it will help with efficiency.
- Spatial Locality (Space Locality)
 - This states that if a memory block is referenced once, the neighboring memory blocks will be accessed too.
 - Since instructions of a single program is stored in contiguous memory.
 - Therefore caching it will help with efficiency.

4.7 Cache Memory

- Resides within the CPU
- Transient - Highly volatile
- May keep frequently accessed data
- May keep data that would be referenced in the future
- There are different levels of caches (L1, L2, L3)
- Cache Performance;

Cache Hit : Requested data for processing by a CPU is found in cache

Cache Hit Ratio : The probability of finding the data for processing in the cache

Cache Miss : Not found in cache

Cache Miss Ratio : 1 - Cache Hit Ratio

Effective Access Time (EAT) : Average time it takes to process data

$$EAT = HR * T_c + (1 - HR) * T_m$$

HR - Hit Ratio

T_c - Cache access time in a cache hit

T_m - Data access time in a cache miss

4.7.1 Key Design Decisions of a Cache

- Cache Volume (Capacity, Size)
- Write Policy
 - Some data may appear on multiple locations and may be shared by multiple processes.
 - Maintaining consistency is a challenge in situations like this.
 - Cache uses write policy to make sure the data stays consistent throughout all the locations.
 - Write Through:
 - * When write operation occurs, it is performed on both main memory and cache synchronously.
 - * Bus traffic will be increased.
 - * Easy to implement.
 - Write Block:
 - * When write operation occurs, it is performed only on cache, but whenever the memory is evicted from cache, it is written to main memory.
 - * Lazy writes.
 - * Complex to implement.
- Mapping Schemes
 - CPU refers data or instruction by addresses.
 - Since cache memory is smaller than main memory, its memory locations cannot be given main memory addresses.
 - Therefore mapping schemes are required.
 - Data is transferred between memory and cache in blocks of fixed size called cache lines (cache blocks), which usually contains multiple bytes or words of data.
 - After copying a cache line, a cache entry is created, which includes copied data as well as the memory location identifier (tag).
 - Amount of data in a cache line and cache entry is always equal.
 - Direct Mapped:
 - * A memory is identified as blocks of memory.
 - * Each block has a predefined cache entry.
 - * Since the maximum number of blocks is always larger than the maximum number of cache entries, multiple blocks match with a same cache entry position.
 - * Tag is used to identify the address of mapped data.
 - Fully Associative:
 - * This is the opposite of direct mapping scheme.
 - * No restrictions are available so cache lines can be placed anywhere.
 - * More flexible cache mapping scheme but searching can be expensive.
 - Set Associative:
 - * Hybrid model of fully associative and direct mapping schemes.
 - * Cache is divided into equal sized sets, and cache line can only be mapped to one set. (Direct mapping)
 - * Within the mapped set, cache line can be placed to anywhere available. (Fully Associative)
 - * When implementing set associative caches, we need to affirm a cache replacement policy to decide which cache entry should be evicted when the set is fully occupied.
 - * Least Recently Used policy : The cache entry which is unused for the longest time will be evicted.
 - * Least Frequently Used policy : The cache line with least access frequency will be evicted.
 - * First in First Out policy : The cache entry which has the longest period of stay will be evicted.
 - * Random Policy